Lab 3: Warps, thread divergence and striding
Geavanceerde Computerarchitectuur

Sien Peerlinck & Thibaut Dalez

March 8, 2024

1 Introduction

This was a lab on warps, thread divergence and striding.

2 Code

2.1 Invert image

First we implemented a kernel for inverting the RGB values of an input value. Each thread calculates the new pixel value for a corresponding pixel in the output image. In case not enough threads were used, striding was used. By using striding, each thread handles multiple pixels. The stride value used equals the total amount of threads, the loop index was each loop incremented by this stride value making sure each pixel was handled without having pixels handled by multiple threads.

The operation performed by each thread is of equal complexity and independent, resulting in no thread divergence.

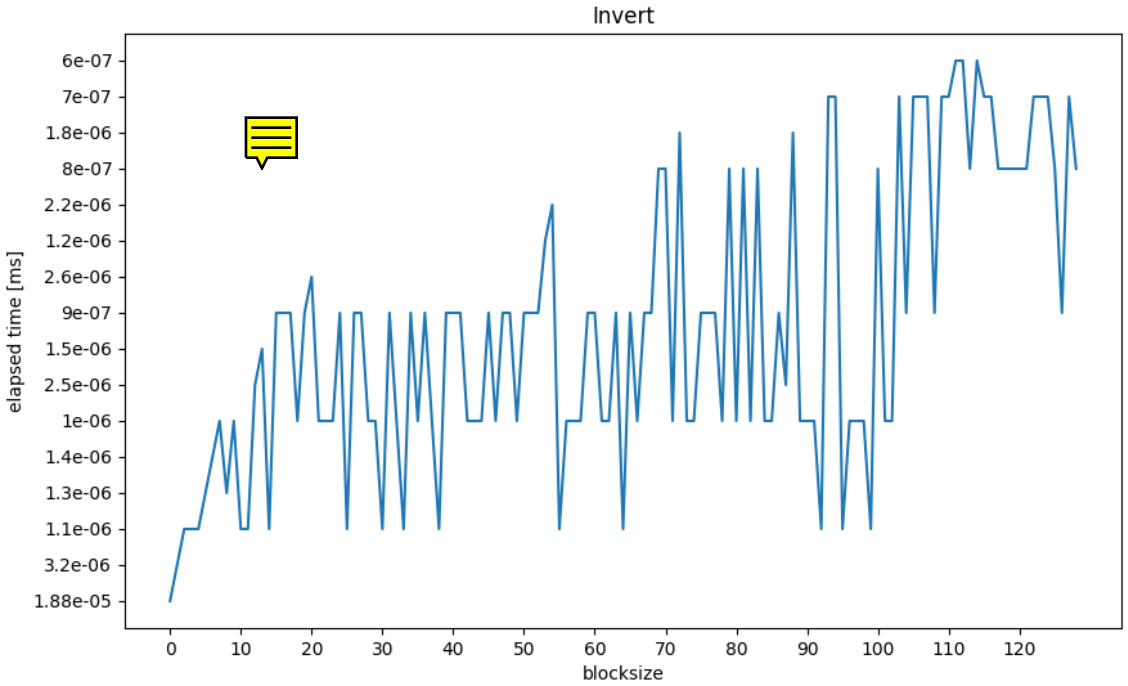


Figure 1: Timing analysis of invert image kernel.

2.2 Divergence

In this kernel, some complexity was added to some of the pixel values, creating thread divergence. The red pixel components with a value between 100 and 200 are calculated as $x = (x\%25) \cdot 10$, every other pixel was again inverted as in the previous kernel. Because of this conditional branching, some threads will have a longer execution time causing the other threads in the warp to run idle leading to thread divergence which can severely impact performance.

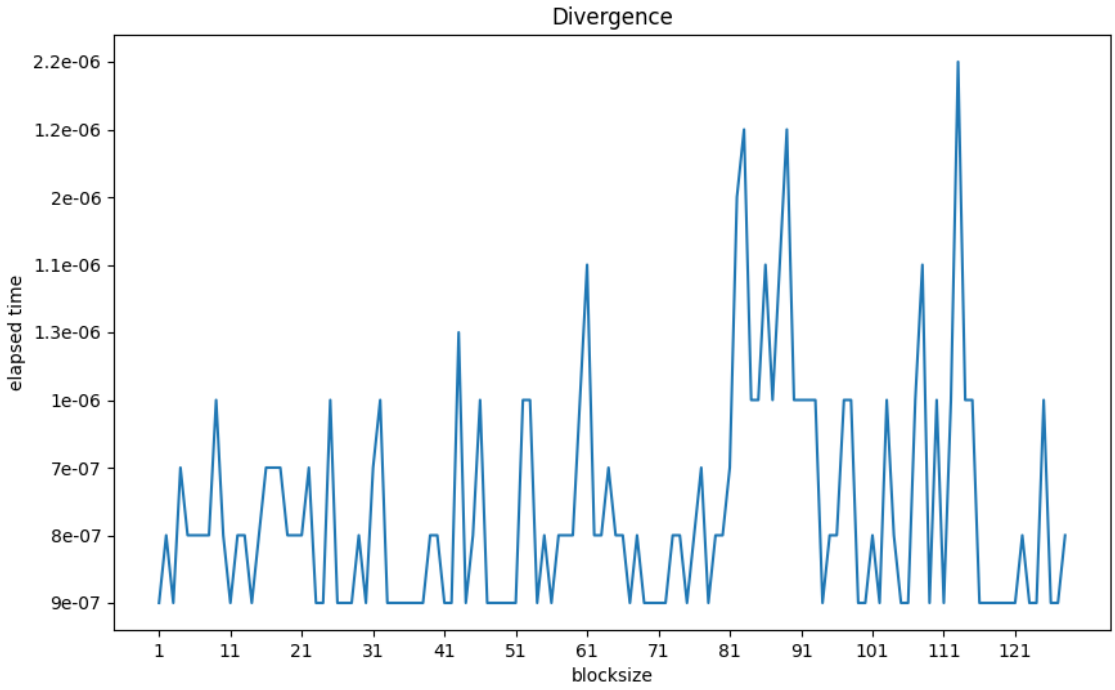


Figure 2: Timing analysis of Divergence kernel.

3 Conclusion

We can conclude from the invert image code that with a bigger blocksize the elapsed time also rises. But when the blocksize is a multiple of 32 it is the lowest (because all warps are fully used, making it an efficient implementation), and n*32 +1 is higher then its neighbours (because full warps are still used, but in the last warp only one thread is actually doing the work, the rest is running idle, making it an inefficient implementation). The reason it keeps getting higher is because we use more cores inefficiently. Because we have a high amount of warps (that each use 32 threads), at some point they use more threads then are actually needed.

The Divergence shows that when we allocate 2 blocks instead of 1 the time is the lowest, this is because there are 2 stream multiprocessors. Otherwise we think this data may be faulty because there is no clear increase/decrease. Normally the time should keep decreasing with the amount of blocks we use because it could be processed more efficiently.

A Program

```
#include <cstdint>      // Data types
#include <iostream>     // File operations
#include <cuda_runtime.h>
#include <chrono>
#include <fstream>

#define M 512       // Lenna width
#define N 512       // Lenna height15
// #define M 941       // VR width
// #define N 704       // VR height
#define C 3         // Colors
#define OFFSET 15   // Header length


__global__ void invert_image(uint8_t *image_array, uint8_t *output_array){
    int tid = blockIdx.x * blockDim.x + threadIdx.x;
    int stride = blockDim.x*gridDim.x;

    for(int i = tid; i<M*N*C; i+=stride){
        output_array[i] = 255 - image_array[i]; //inverting the rgb-values
    }
}

__global__ void divergent_image(uint8_t *image_array, uint8_t *output_array){
    int tid = blockIdx.x * blockDim.x + threadIdx.x;
    int stride = blockDim.x*gridDim.x;

    for(int i = tid; i<M*N*C; i+=stride){
        if(tid%3==0 && 100<image_array[i]<200){
            output_array[i] = (image_array[i]%25)*10;
        }
        else{
            output_array[i] = 255 - image_array[i];
        }
    }


}


uint8_t* get_image_array(void){
    /*
     * Get the data of an (RGB) image as a 1D array.
     *
     * Returns: Flattened image array.
     *
     * Noets:
     *  - Images data is flattened per color, column, row.
     *  - The first 3 data elements are the RGB componentsinvert_image
     *  - The first 3*M data elements represent the firts row of the image
     *  - For example, r_{0,0}, g_{0,0}, b_{0,0}, ..., b_{0,M}, r_{1,0}, ..., b_{b,M}, ..., b_
    {N,M}
     *
     */
    // Try opening the file
    FILE *imageFile;
    imageFile=fopen("./input_image.ppm","rb");
    if(imageFile==NULL){
        perror("ERROR: Cannot open output file");
        exit(EXIT_FAILURE);
    }

    // Initialize empty image array
    uint8_t* image_array = (uint8_t*)malloc(M*N*C*sizeof(uint8_t)+OFFSET);

    // Read the image
    fread(image_array, sizeof(uint8_t), M*N*C*sizeof(uint8_t)+OFFSET, imageFile);

    // Close the file
    fclose(imageFile);

    // Move the starting pointer and return the flattened image array
    return image_array + OFFSET;
}

```

```
75
76 void save_image_array(uint8_t* image_array){
77     /*
78      * Save the data of an (RGB) image as a pixel map.
79      *
80      * Parameters:
81      *   - param1: The data of an (RGB) image as a 1D array
82      *
83      */
84     // Try opening the file
85     FILE *imageFile;
86     imageFile=fopen("./output_image.ppm","wb");
87     if(imageFile==NULL){
88         perror("ERROR: Cannot open output file");
89         exit(EXIT_FAILURE);
90     }
91
92     // Configure the file
93     fprintf(imageFile,"P6\n");               // P6 filetype
94     fprintf(imageFile,"%d %d\n", M, N);      // dimensions
95     fprintf(imageFile,"255\n");              // Max pixel
96
97     // Write the image
98     fwrite(image_array, 1, M*N*C, imageFile);
99
100    // Close the file
101    fclose(imageFile);
102 }
103 void save_image_array_divergent(uint8_t* image_array){
104     /*
105      * Save the data of an (RGB) image as a pixel map.
106      *
107      * Parameters:
108      *   - param1: The data of an (RGB) image as a 1D array
109      *
110      */
111     // Try opening the file
112     FILE *imageFile;
113     imageFile=fopen("./output_image2.ppm","wb");
114     if(imageFile==NULL){
115         perror("ERROR: Cannot open output file");
116         exit(EXIT_FAILURE);
117     }
118
119     // Configure the file
120     fprintf(imageFile,"P6\n");               // P6 filetype
121     fprintf(imageFile,"%d %d\n", M, N);      // dimensions
122     fprintf(imageFile,"255\n");              // Max pixel
123
124     // Write the image
125     fwrite(image_array, 1, M*N*C, imageFile);
126
127     // Close the file
128     fclose(imageFile);
129 }
130
131 int main (void) {
132
133     // Read the image
134     uint8_t* image_array = get_image_array();
135
136     // Allocate output
137     uint8_t* output_array = (uint8_t*)malloc(M*N*C*sizeof(uint8_t));
138
139
140
141     uint8_t *d_image_array;
142     cudaMalloc(&d_image_array, M*N*C*sizeof(uint8_t));
143     cudaMemcpy(d_image_array, image_array, M*N*C*sizeof(uint8_t), cudaMemcpyHostToDevice);
144
145     uint8_t *d_output_array;
146     cudaMalloc(&d_output_array, M*N*C*sizeof(uint8_t));
147
148     int blockSize = 64;
149     int gridSize = 1;
150
```

```
    std::ofstream outputFile("timing_invert.csv"); // for the measurements
    if (!outputFile.is_open()) {
        std::cerr << "Error opening file!" << std::endl;
        return 1;
    }

    outputFile << "blocksize,elapsed_time" << std::endl;

    for(int i = 0; i <= 128; i += 1) {
        const auto startinvert = std::chrono::high_resolution_clock::now();

        invert_image<<<gridSize, import>>>(d_image_array, d_output_array);  // geef pointers
    naar arrays mee
        const auto endinvert = std::chrono::high_resolution_clock::now();

        cudaMemcpy(output_array, d_output_array, M*N*C*sizeof(uint8_t), cudaMemcpyDeviceToHost
    ); // cpu memory to GPU memory
        cudaFree(d_output_array);

        const std::chrono::duration<double> elapsed_seconds_invert{endinvert - startinvert};

        outputFile << i << "," << elapsed_seconds_invert.count() << std::endl;
    }
    outputFile.close();


    std::ofstream outputFile("timing_divergent.csv"); // for the measurements
    if (!outputFile.is_open()) {
        std::cerr << "Error opening file!" << std::endl;
        return 1;
    }

    outputFile << "blocksize,elapsed_time" << std::endl;

    uint8_t* divergent_array = (uint8_t*)malloc(M*N*C*sizeof(uint8_t));
    uint8_t *d_divergent_array;
    cudaMalloc(&d_divergent_array, M*N*C*sizeof(uint8_t));

    for(int i = 0; i <= 128; i += 1) {

        const auto startdivergent = std::chrono::high_resolution_clock::now();

        // divergent_image<<<gridSize, blockSize>>>(d_image_array, d_divergent_array);  //
    geef pointers naar arrays mee
        divergent_image<<<gridSize, M*N*C>>>(d_image_array, d_divergent_array);  // geef
    pointers naar arrays mee
        cudaMemcpy(divergent_array, d_divergent_array, M*N*C*sizeof(uint8_t),
    cudaMemcpyDeviceToHost); // cpu memory to GPU memory

        cudaFree(d_image_array);
        cudaFree(d_divergent_array);


        const auto endivergent = std::chrono::high_resolution_clock::now();
        const std::chrono::duration<double> elapsed_seconds_divergent{endivergent -
    startdivergent};
        outputFile << i << "," << elapsed_seconds_divergent.count() << std::endl;
    }
    outputFile.close();

    std::cout << elapsed_seconds_invert.count() << "\n";
    save_image_array(output_array);
    save_image_array_divergent(divergent_array);
    return 0;
}
```

Listing 1: Image inversion